```
from sklearn.model_selection import train_test_split
import numpy as np
```

```
experience=np.array([1,2,3,4,5,6,7,8,9,10,1,12,13,14,15])
salary=np.array([35,38,42,48,55,60,65,70,76,82,87,92,96,100,105])
x=experience.reshape(-1,1)
y=salary.reshape(-1,1)
x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.2,random_state=42)
print(f"Training samples:{len(x_train)}")
print(f"Testing samples:{len(x_test)}")
```

```
Training samples:12
Testing samples:3
```

```
from sklearn.model_selection import train_test_split
import numpy as np
```

```
experience = np.array([1,2,3,4,5,6,7,8,9,10,11,12,13,14,15])
salary = np.array([35,38,42,48,55,60,65,70,76,82,87,92,96,100,105])
```

```
X = experience.reshape(-1, 1) # sklearn expects 2D array for features
y = salary
```

```
X_train, X_test, y_train, y_test = train_test_split(
 X, y,
 test_size=0.2, # 20% for testing
 random_state=42 # ensures same split every run
)
```

```
print(f"Training samples : {len(X_train)}")
print(f"Testing samples : {len(X_test)}")
```

```
Training samples : 12
Testing samples : 3
```

```
from sklearn.linear_model import LinearRegression
```

```
model = LinearRegression()
```

```
model.fit(X_train, y_train)
```

```
▾ LinearRegression
LinearRegression()
```

```
print(f"Slope (coefficient) : {model.coef_[0]:.2f}")#slope=coef
print(f"Intercept : {model.intercept_:.2f}")
```

```
Slope (coefficient) : 5.22
Intercept : 27.94
```

```
y_pred = model.predict(X_test)
```

```
for actual, predicted in zip(y_test, y_pred):
 print(f"Actual: {actual:>5} | Predicted: {predicted:>7.1f} | "
 f"Error: {predicted - actual:+.1f}")
```

```
Actual:    82 | Predicted:    80.2 | Error: -1.8
Actual:    92 | Predicted:    90.6 | Error: -1.4
Actual:    35 | Predicted:    33.2 | Error: -1.8
```

```
from sklearn.metrics import r2_score, mean_absolute_error, mean_squared_error
import numpy as np
```

```
y_actual = np.array([500, 300, 700, 450, 600])
y_pred = np.array([480, 320, 690, 460, 590])
```

```
r2 = r2_score(y_actual, y_pred)
```

```
mae = mean_absolute_error(y_actual, y_pred)
```

```
rmse = np.sqrt(mean_squared_error(y_actual, y_pred))
print(f"R² Score : {r2:.4f} → model explains {r2*100:.1f}% of variance")
print(f"MAE : {mae:.1f} → avg error ± {mae:.0f} units")
print(f"RMSE : {rmse:.1f} → penalised avg error {rmse:.0f} units")
```

```
R² Score : 0.9880 → model explains 98.8% of variance
MAE : 14.0 → avg error ± 14 units
RMSE : 14.8 → penalised avg error 15 units
```

```
import pandas as pd
import numpy as np
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import r2_score, mean_absolute_error
import numpy as np
```

```
np.random.seed(42)
n = 100
data = pd.DataFrame({
 'experience': np.random.randint(0, 20, n),
 'education': np.random.randint(12, 20, n),
 'age': np.random.randint(22, 55, n),
})
```

```
data['salary'] = (
 data['experience'] * 4500 +
 data['education'] * 2000 +
 data['age'] * 300 +
 np.random.normal(0, 3000, n) +
 10000
)
```

```
X = data[['experience', 'education', 'age']]
y = data['salary']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
model = LinearRegression()
model.fit(X_train, y_train)
```

```
▾ LinearRegression
LinearRegression()
```

```
for feat, coef in zip(X.columns, model.coef_):
 print(f" {feat:12s}: ${coef:,.0f} per unit increase")
print(f" Base salary (intercept): ${model.intercept_:,.0f}")
y_pred = model.predict(X_test)
print(f"\nR² : {r2_score(y_test, y_pred):.3f}")
print(f"MAE : ${mean_absolute_error(y_test, y_pred):,.0f}")
```

```
 experience  : $4,534 per unit increase
 education   : $2,203 per unit increase
 age         : $266 per unit increase
 Base salary (intercept): $7,694

R² : 0.988
MAE : $2,219
```